

DataMaster

Repository context and documentation scope

A concise summary of what the prototype does, how its major parts fit together, and the confirmed boundary for the current documentation work.

INTERNAL DOCUMENTATION ONLY

Decision

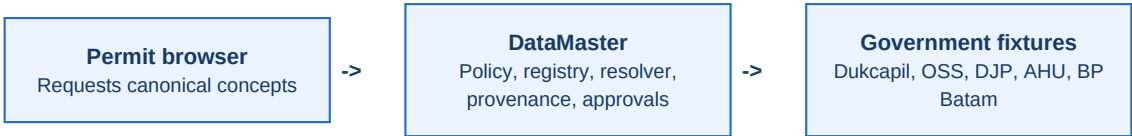
No further API-documentation changes are proposed for this repository. The current scope updates internal explanatory documentation only. Existing OpenAPI contracts, generated API reference material, endpoint behavior, schemas, and runtime code remain unchanged.

Product	DataMaster - a government integration control layer demonstrated through a Batam logistics-permit scenario.
Primary audience	Judges, developers, reviewers, and operators evaluating how government integrations are discovered, governed, and safely evolved.
Technology	Python, FastAPI, deterministic mock government services, JSON-backed demo state, OpenAPI 3.1 contracts, and optional LLM-assisted proposals.
Maturity	A local, judge-facing prototype. It deliberately omits production IAM, durable transactional storage, national-scale audit infrastructure, and multi-node operations.
Prepared	15 August 2026, Asia/Jakarta.

Interpretation note: this brief describes the DataMaster repository that was inspected. It is separate from the ongoing Residential App project.

1. Product and system context

DataMaster demonstrates how a new public-service workflow can reuse verified records that government agencies already hold. Its demo follows PT Selat Niaga Makmur: the company previously completed a BP Batam land-allocation process and later applies for a logistics business permit. The permit should reuse identity, tax, company, licensing, and land records while leaving genuinely new application evidence for manual entry.



The stable boundary

The main workflow calls `POST /api/resolve` with canonical concepts such as `business.nib`. DataMaster resolves those concepts through approved registry mappings, calls the relevant government fixtures, and returns per-field provenance plus a routing trace. The permit-facing request can remain stable even when an upstream provider changes its path or field names.

Governance and change control

- A deterministic purpose policy blocks disallowed concepts before any source call. The permit flow specifically demonstrates that health data is not queried.
- A legacy BP Batam LMS fixture can be scanned with Python's AST parser. DataMaster proposes a constrained adapter and related contract artifacts, runs checks, and waits for human approval.
- OSS v1 and v2 OpenAPI contracts are structurally diffed. A dependency graph identifies affected concepts and consumers before a mapping change can be approved.
- AI assistance is proposal-only. Authorization, structural diffing, fixture checks, contract tests, approval, and activation remain deterministic gates.

Implemented surfaces

Resolution	Canonical concept resolution, provenance, and stable permit integration.
Onboarding	AST scan, adapter proposal, checks, persisted review state, and human approval.
Contract change	OpenAPI diff, impact analysis, candidate mapping, deterministic tests, approval, and registry activation.
Operations	Demo reset, isolated automated checks, local launch script, mock upstream services, and optional webhook-shaped change intake.

Security posture

The prototype contains important safety ideas - purpose limitation, data minimization, provenance, deterministic approval gates, and optional webhook HMAC verification - but it is not production-secure. It has no production IAM, client scopes, durable database, managed secrets, encryption integration, rate limiting, or tamper-evident audit log.

2. Confirmed documentation scope

The phrase 'internal documentation only' is a change boundary. It allows the repository's explanatory material to be clarified without changing any public contract or runtime behavior.

AREA	STATUS	WHAT THIS MEANS
Internal repository documentation	IN SCOPE	Clarify the project purpose, current architecture, operational model, security limitations, and the distinction between canonical resolution and API-contract management.
API contracts and schemas	NO CHANGE	Do not edit the OSS v1/v2 OpenAPI files, endpoint shapes, request or response schemas, parameters, status codes, or version identifiers.
Generated API reference material	NO CHANGE	Do not propose or regenerate OpenAPI-derived Markdown, the legacy LMS OpenAPI proposal, or other consumer-facing API reference artifacts as part of this work.
Runtime behavior	NO CHANGE	No routes, resolver mappings, policy rules, approval gates, service registry entries, fixtures, or tests are changed by this documentation task.

Important distinction

DataMaster's product behavior still includes OpenAPI ingestion, structural comparison, a legacy-adapter OpenAPI proposal, and OpenAPI-derived documentation generation. Those capabilities remain part of the demo. The present decision simply means this documentation task does not add, revise, or regenerate API documentation.

Existing pending demo proposal

At inspection time, `state/proposals.json` contained one pending legacy LMS onboarding proposal. It includes a generated OpenAPI 3.1 proposal and a generated documentation preview. This is existing demo state, not a new documentation change requested by this brief. The internal-documentation-only decision neither approves, rejects, edits, nor removes that proposal.

What internal documentation should make clear

- The current primary resolver is deterministic and centered on canonical concepts and purpose policy.
- OpenAPI describes individual HTTP API contracts; DataMaster manages mappings, dependencies, impact analysis, provenance, approvals, and integration lifecycle across services.
- AI output is advisory and must not silently activate production behavior.
- Local mock services, JSON state, and the fixed demo reviewer are simulations, not production architecture.
- Consumer applications should not receive government credentials; production calls should use authenticated server-to-server boundaries.

Internal distinction to make explicit

The current `README.md` describes the stable `POST /api/resolve` architecture and deterministic controls. The supplemental `docs/MIDDLEWARE.md` emphasizes `POST /api/query`, free-form field names, and LLM-driven tool routing. Both endpoints are present in `app/main.py`. Internal documentation should distinguish the stable production-oriented resolver from the optional LLM Playground contract so readers do not assume they provide the same guarantees. This is a prose clarification; it does not require an API change.

3. Documentation-only acceptance criteria

The scope can be considered correctly documented when all of the following statements are true.

- Repository explanations consistently describe DataMaster as a government integration control layer for the Batam permit demonstration.
- The distinction between stable canonical resolution and upstream API-contract management is explicit.
- Internal documents clearly distinguish `/api/resolve` from `/api/query` and do not present optional LLM routing as equivalent to the stable resolver contract.
- No file under `contracts/` or `generated/openapi/` is changed for this work.
- No OpenAPI-derived reference document is proposed or regenerated for this work.
- No request schema, response schema, route, mapping, registry entry, policy rule, approval gate, fixture, or test behavior is modified.
- The existing pending legacy LMS proposal is clearly treated as demo state, not as part of the internal-documentation update.

Repository evidence used for this summary

README.md	Current product narrative, architecture, demo workflow, endpoints, limitations, and production path.
docs/MIDDLEWARE.md	Supplemental <code>/api/query</code> middleware narrative and integration examples that require clear scope labeling.
app/main.py	FastAPI boundary and current judge-facing routes.
app/onboarding.py	Legacy scan, adapter proposal, OpenAPI proposal, checks, and approval flow.
app/change_manager.py	OpenAPI diff, dependency impact, mapping approval, tests, activation, and documentation regeneration behavior.
state/*.json	Current service registry, concept mappings, demo state, and pending proposal state.
contracts/*.openapi.json	Existing OSS v1 and v2 API contracts that remain outside the current change scope.

Final scope statement

No API-documentation proposal

This work is limited to internal repository documentation. It records and clarifies DataMaster's context; it does not propose changes to API documentation, OpenAPI contracts, generated reference artifacts, endpoint behavior, or runtime implementation.

This summary is based on the repository snapshot inspected on 15 August 2026. It is an engineering documentation brief, not legal or production-readiness certification.